

Optimización de consultas SQL

Cómo encontrar la consulta lenta, entender por qué es lenta y corregirla. Todos los ejemplos son para MySQL 8 con InnoDB.

Tablas de ejemplo: clientes (id, nombre, email) y pedidos (id, cliente_id, estado, total, creado_en).

1 Lee el plan con EXPLAIN

Antes de cambiar nada, pregúntale a MySQL cómo piensa ejecutar la consulta.

```
EXPLAIN
SELECT id, total FROM pedidos
WHERE cliente_id = 42;
```

COLUMNA	QUÉ MIRAR
type	Cómo accede a la tabla (ver escala abajo)
key	Índice que usa. NULL = ninguno
rows	Filas que calcula que va a revisar
Extra	Using filesort o Using temporary = alerta

ALL

index

range

ref

eq_ref

const

Peor: recorre toda la tabla

Mejor: una sola fila

EXPLAIN ANALYZE (8.0.18 o superior) ejecuta la consulta y muestra tiempos reales, no estimados.

3 Índices compuestos: el prefijo izquierdo

Un índice de varias columnas sirve de izquierda a derecha, como un directorio ordenado por apellido y luego por nombre.

```
CREATE INDEX idx_cliente_estado_fecha
ON pedidos (cliente_id, estado, creado_en);
```

WHERE	¿USA EL ÍNDICE?
cliente_id = 42	Sí
cliente_id = 42 AND estado = 'pagado'	Sí
estado = 'pagado'	Casi nunca

Orden recomendado: primero columnas con igualdad (=), al final la columna de rango (>, <, BETWEEN).

2 Indexa donde filtras y unes

Las columnas que aparecen en WHERE, en JOIN ... ON y en ORDER BY son las candidatas a índice.

```
CREATE INDEX idx_pedidos_cliente
ON pedidos (cliente_id);

-- Ver los índices de una tabla
SHOW INDEX FROM pedidos;
```

Cada índice hace más lentos los INSERT y UPDATE y ocupa disco. Crea los que tus consultas usan, no uno por columna.

4 No escondas la columna en una función

Si la columna indexada queda dentro de una función o de un cálculo, MySQL no puede usar el índice y revisa fila por fila.

EVITA

```
WHERE YEAR(creado_en) = 2025
```

MEJOR

```
WHERE creado_en >= '2025-01-01'
AND creado_en < '2026-01-01'
```

EVITA

```
WHERE email LIKE '%@gmail.com'
```

MEJOR

```
WHERE email LIKE 'ana%'
```

Lo mismo pasa al comparar tipos distintos: si telefono es VARCHAR, escribe telefono = '7025550100' con comillas.

5 Pide solo las columnas que usas

SELECT * trae datos que tu código no lee. Si todas las columnas pedidas están en el índice, MySQL ni siquiera abre la tabla: EXPLAIN muestra Using index en Extra.

EVITA

```
SELECT * FROM pedidos WHERE cliente_id = 42;
```

MEJOR

```
-- Con INDEX (cliente_id, estado, total)
SELECT estado, total
FROM pedidos
WHERE cliente_id = 42;
```

Ese índice se llama índice de cobertura.

7 Evita el problema N+1

Una consulta para la lista y otra por cada fila dentro de un bucle. Con 200 clientes son 201 viajes a la base de datos.

EVITA

```
-- 1 vez
SELECT id, nombre FROM clientes;
-- 1 vez POR CADA cliente, en el bucle PHP
SELECT COUNT(*) FROM pedidos WHERE cliente_id = ?;
```

MEJOR

```
SELECT c.id, c.nombre, COUNT(p.id) AS pedidos
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
GROUP BY c.id, c.nombre;
```

6 Pagina con keyset, no con OFFSET

Con OFFSET, MySQL lee todas las filas anteriores y luego las descarta. En la página 5000 eso son 100 000 filas tiradas.

EVITA

```
SELECT id, total FROM pedidos
ORDER BY id DESC
LIMIT 20 OFFSET 100000;
```

MEJOR

```
SELECT id, total FROM pedidos
WHERE id < 250310 -- último id mostrado
ORDER BY id DESC
LIMIT 20;
```

La desventaja: no puedes saltar directo a la página 37. Sirve para "cargar más" y scroll infinito.

8 Para saber si existe, no cuentes

COUNT(*) revisa todas las filas que coinciden. EXISTS se detiene en la primera.

EVITA

```
SELECT COUNT(*) FROM pedidos
WHERE cliente_id = 42; -- y luego: if ($n > 0)
```

MEJOR

```
SELECT EXISTS (
  SELECT 1 FROM pedidos WHERE cliente_id = 42
) AS tiene_pedidos;
```

Devuelve 1 o 0.

9 Revisión antes de subir a producción

- ☐ ¿Corriste EXPLAIN en las consultas nuevas?
- ☐ ¿Alguna tabla grande aparece con type = ALL?
- ☐ ¿Hay una función o un cálculo sobre una columna del WHERE?
- ☐ ¿Queda algún SELECT * que se pueda reducir?
- ☐ ¿Hay una consulta dentro de un bucle de PHP?
- ☐ ¿Alguna paginación usa un OFFSET que puede crecer sin límite?

```
-- Índices que nadie usa desde el último reinicio del servidor
SELECT * FROM sys.schema_unused_indexes
WHERE object_schema = 'tu_base_de_datos';
```